

Containerlab & Network Automation — an Aston Tech Course

FOR
DUMMIES

From “empty vCenter pool” to “a Clos fabric that configures itself.”

This course teaches two things at once, because they’re better learned together:

1. Containerlab — network topologies as code. Real Arista EOS, deployed in seconds, torn down without guilt, version-controlled like any other code.
2. Network automation — Netmiko → Nornir → NAPALM, from “SSH one device” to declarative, idempotent, template-driven config — the same arc as Aston’s original automation course, now running against a lab you built yourself instead of GNS3 behind a jump host.

No prior Containerlab or automation experience assumed. You should be comfortable in a Linux shell and know what OSPF and BGP are.

The modules

Work through them in order — each builds on the last.

Module	Title	You walk away with
00	Orientation: the map	Knowing what every file in this repo is for, and a decision tree for “I want to change X → touch Y → run Z”
01	The platform	A Debian VM in Aston’s vCenter running Docker + Containerlab, built via <code>setup.sh</code>
02	Containerlab basics	The 2-node lab deployed, understood, broken, fixed, and destroyed — the full lab lifecycle
03	The spine-leaf fabric	A 4-node Clos fabric with an OSPF underlay you configured <i>by hand</i> — so you feel the pain automation removes
04	Automation	The same underlay deployed by one idempotent script; then your first

Module	Title	You walk away with
		bespoke Nornir script
05	VXLAN + EVPN overlay	L2 stretched between leaves over the routed fabric; capstone: automate it yourself
06	Secrets (optional)	The four credential-management approaches in this repo, compared, and when each earns its keep

Time budget: modules 01–02 in an afternoon. 03–04 in another. 05 is a half day if EVPN is new to you. 06 is an hour whenever.

Ground rules that make this go well

Destroy your labs when you stop for the day. cEOS nodes eat 0.5–1 GB RAM each and the VM doesn’t get bigger. write memory on the nodes, then destroy — Containerlab keeps the startup-configs, and the next deploy boots right back into them. This is the single biggest habit shift coming from GNS3, where labs run for weeks.

Type the verification commands. Every module has checkpoints. They aren’t decoration — when module 05 doesn’t work, it’s almost always because a module 03 checkpoint got skipped.

When you’re lost, go back to [Module 00](#). The file map and decision tree exist precisely for the moment where you’re staring at the repo thinking “wait, which YAML do I edit for this?”

The one command to remember

Between modules, `./lab` at the repo root drives whatever you’ve deployed:

```
./lab          # menu, if you'd rather not memorise verbs
./lab status   # what's running
./lab ssh      # log into a node (user admin, password admin)
./lab graph    # topology diagram in a browser
./lab destroy  # tear it down before you stop for the day
./lab learn    # this module list, from the shell
```

It’s a convenience wrapper, not a replacement for the modules — the course teaches the raw containerlab commands on purpose, and you should be able to type them. But when you’ve finished a module and just want to poke at the lab, this saves a trip back to the docs.

Two ways to drive

`setup.sh` at the repo root automates the whole platform build and is the right choice for module 01. From module 02 onward, this course deliberately uses the manual commands — the point is to learn what the wrapper is doing for you, not to press Enter until a network appears. Everything the wrapper does maps to a numbered script in `scripts/` or a file you’ll meet in module 00.